

Отчёт по разработке подходов В и С к реализации файловой системы

Проект: эмпирическая модель производительности in-
memory файловой системы

Содержание

1. Цели и задачи разработки	2
2. Краткий обзор требований к FS и интерфейса IFileSystem	2
3. Базовые утилиты: MemoryPool и PathUtils	3
4. Система оценки памяти	5
5. Подход В: Дерево + Хэш-таблица	6
6. Подход С: Плоское хэш-отображение	6

1. Цели и задачи разработки

- **Цель:** разработка высокопроизводительных реализаций in-memory файловой системы (подходы В и С) для последующего сбора метрик и обучения эмпирических моделей.
 - **Задачи:**
 1. Проектирование и интеграция кастомного аллокатора `MemoryPool` для оптимизации скорости создания узлов и честного подсчёта системной памяти.
 2. Разработка модуля `PathUtils` для эффективного парсинга путей
 3. Реализация подхода В (Дерево + Хэш-таблица) и подхода С (Плоское хэш-отображение) по заданному интерфейсному классу `IFileSystem`
-

2. Краткий обзор требований к FS и интерфейса `IFileSystem`

- **Ограничения файловой системы:** система не является полноценной *POSIX-совместимой*. В ней отсутствует архитектурное понятие «*текущей рабочей директории*» (*CWD*), поэтому она поддерживает только абсолютные пути. Полный отказ от относительных переходов (`.` и `..`) и сложной нормализации путей позволяет *избежать коллизий* в хэш-таблицах.
- **Интерфейс `IFileSystem`:** все подходы реализуют единый общий интерфейс `IFileSystem`, спроектированный другим участником. Это гарантирует, что бенчмарк запускает одинаковую нагрузку и взаимодействует с любой реализацией абсолютно идентично:

- **Операции:** реализована поддержка операций чтения (read), записи (write), создания директорий (mkdir), листинга (ls), перемещения (mv) и поиска (find). Также присутствуют операции проверки (exists), очистки системы (clear) и получения состояния узла/системы.
-

3. Базовые утилиты: MemoryPool и PathUtils

- **Memory Pool (Free-list аллокатор):**

1. Обоснование необходимости:

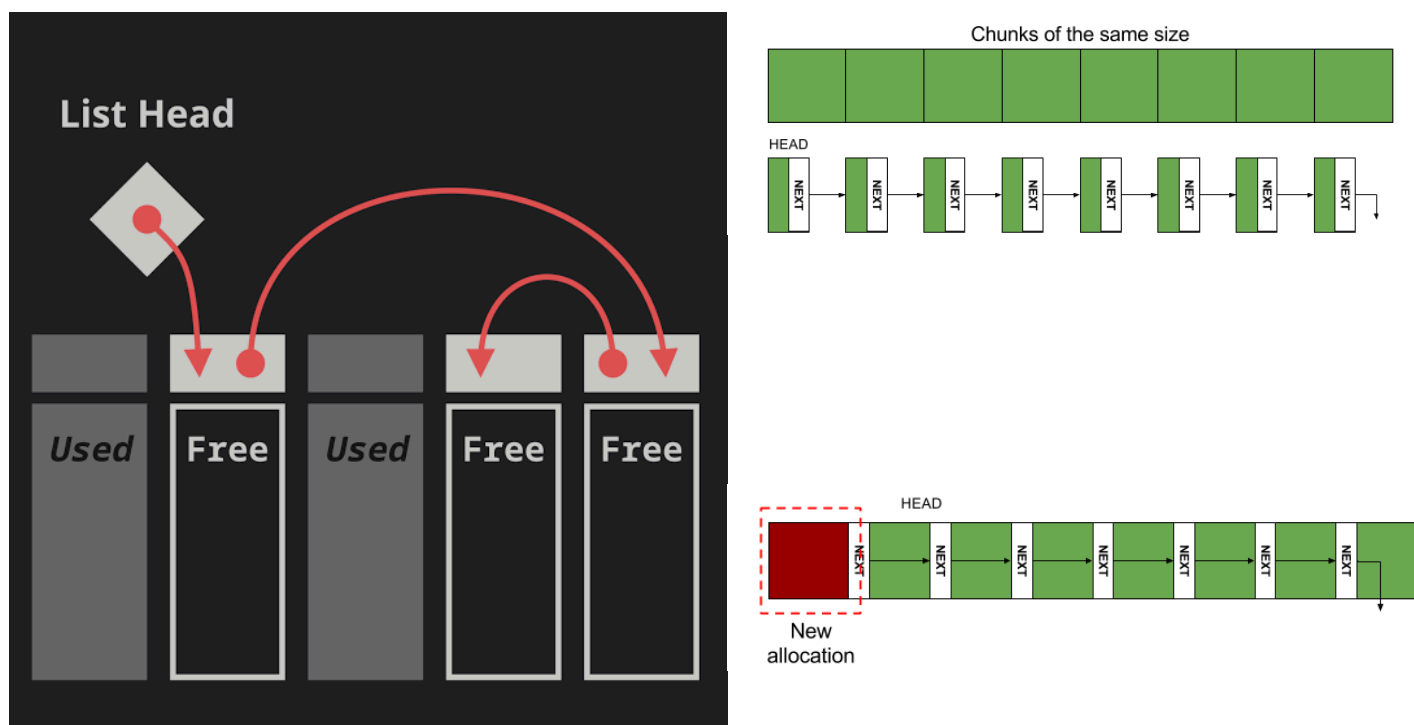
- ▶ Любой подход опирается на требование динамической аллокации структур узла файловой системы (`struct Node` или `struct FsNode`)
- ▶ Постоянные системные вызовы при использовании оператора `new` для создания новых узлов файловой системы сильно фрагментируют память и замедляют не только непосредственную работу бенчмарка, но и подготовительный этап (создания файловой системы по заданным параметрам).

Вывод: для ускорения проведения бенчмарков требуется реализовать кастомный аллокатор, который решает данные проблемы

2. Описание работы Free-List аллокатора:

- ▶ Аллокатор заранее запрашивает память у ОС большими блоками (чанками), кардинально ускоряя операции write и mkdir.
- ▶ При удалении файла память не возвращается ОС, а добавляется в связный список свободных блоков. Это гарантирует выделение и освобождение памяти на структуру узла за $O(1)$

- ▶ Целенаправленно отказались от STL-совместимостимого аллокатора, так как пул рассчитан только на объекты одного типа с параметризуемым фиксированным размером.



• Утилиты PathUtils:

1. Обоснование необходимости:

- ▶ Вся логика валидации и работы со строками вынесена в статические методы для соблюдения принципа единственной ответственности (Single responsibility principle)
- ▶ Строками с путями владеют узлы файловой системы (`struct Node` или `struct FsNode`), поэтому для избавления от лишних аллокаций в куче, замедляющих работу бенчмарка, используем `std::string_view`

2. Описание реализованных методов:

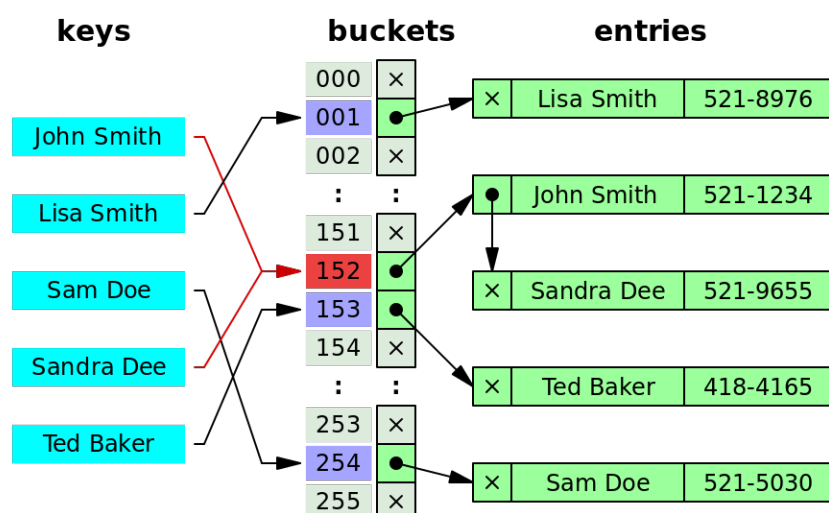
- ▶ Проверка валидности путей согласно принятым ограничениям
- ▶ Безопасное получение родительской директории и имени узла файловой системы

- Разбиение абсолютного пути на компоненты и соединение частей пути
- Поддержка пользовательских glob-паттернов через трансляцию в эквивалентное регулярное выражение и последующую проверку через `std::regex_match`

4. Система оценки памяти

- **Проблема:** стандартный подход подсчета через `sizeof` показывает только «полезную» память, игнорируя реальные накладные расходы и фрагментацию чанков.
- **Решение:** разделение зон ответственности — пул памяти отслеживает выделенную сырую системную память (чанки), а класс файловой системы — объем полезных данных файлов.
- **Формулы расчёта `size_t total_size_bytes`:**

`total_size_bytes` = Объем чанков пула + вес полезных данных + эвристический оверхед хэш-таблицы `std::unordered_map` (массив бакетов из указателей и связанные цепочки из узлов мапы)



5. Подход В: Дерево + Хэш-таблица

- **Структура:** Основная структура остается N-арным деревом, но дополняется хэш-таблицей

`path_index_ = std::unordered_map<std::string_view, FsNode*>`

отображающей абсолютный путь на узел.

- **Вспомогательные методы:**
 - Внедрены приватные методы `GetNode`, `GetFile`, `GetDirectory`, `GetParentDirectory` и фабричный `CreateNewNode` для инкапсуляции поиска по хэш-таблице и валидации типов, что избавило код от бойлерплейта.
 - Внедрены методы инициализации и очистки файловой системы `InitRoot` и `DestroyTree`
 - Добавлен вспомогательный метод рекурсивного обновления путей при перемещении директории `UpdatePathsRecursive`
- **Асимптотика и компромиссы:**
 - **Плюсы:** точечный доступ (`read`, `write`, `mkdir`, `exists`) работает за $O(1)$.
 - **Минусы:** операция перемещения (`mv`) директории имеет сложность $O(N)$, так как требует рекурсивного обхода поддерева для обновления полных путей и перестроения ключей в мапе. Также хранение полных путей увеличивает потребление ОЗУ.

6. Подход С: Плоское хэш-отображение

- **Структура:** Полный отказ от древовидной структуры. Узлы не хранят указатели на родителей и списки детей. Вся фай-

ловая система эмулируется префиксами путей внутри единой хэш-таблицы.

```
path_index_ = std::unordered_map<std::string_view, FsNode*>
```

- **Вспомогательные методы:**

- ▶ Внедрены приватные методы `GetNode`, `GetFile`, `GetDirectory`, `GetParentDirectory` и фабричный `CreateNewNode` для инкапсуляции поиска по хэш-таблице и валидации типов, что избавило код от бойлерплейта.
- ▶ Внедрён метод инициализации файловой системы `InitRoot`, а очистка автоматическая в силу RAII классов хэш-таблицы и пула.

- **Асимптотика и компромиссы:**

- ▶ **Плюсы:** Экстремальная экономия памяти и точечный доступ (`read`, `write`, `mkdir`, `exists`) за $O(1)$.
- ▶ **Минусы:** Катастрофическое падение производительности для операций `ls`, `mv` и `find`. Из-за отсутствия связей между узлами эти методы требуют полного сканирования всей хэш-таблицы за $O(N)$ для поиска совпадений по строковым префиксам.